

My Approach to the Garment Segmentation and Placement Assessment

When I read the brief, the problem became clear. Your current system places artwork at fixed pixel coordinates, so "left chest" always maps to the same x and y regardless of how the garment actually sits in the frame. It has no concept of "this is the chest panel" or "this is the sleeve." It is just guessing coordinates.

So, two things were requested. First, a small model that looks at a garment photo and segments it into body and sleeve regions. Second, a deterministic function that takes that segmentation, an artwork file, and an instruction like "front, left chest," and works out exactly where to place the artwork so it stays on the right part of the garment and never crosses a seam or lands on a sleeve. It had to run on under two million trainable parameters.

There was a bit of confusion as regards the dataset with the email and the brief that came with it and this informed my earlier decisions.

Dataset search: Kaggle, dead ends, and Fashionpedia

I thought from the brief at first that I was mandated to use Fashionpedia and tried downloading it manually from the GitHub since it was not provided but I was having issues with the annotations, I spent a while trying to figure that out and decided to use the Kaggle People Clothing Segmentation dataset instead. It is small, easy to download, and has pixel-level masks for fifty-nine categories. It was a reasonable place to explore without waiting on anything.

There was an issue with the dataset, there was no sleeve category but there was skin, so my first idea was to infer sleeves from skin. I grew the skin mask outward by a few pixels and treated overlapping garment pixels on the sides of the body as sleeve. I ran it on a batch of images and evaluated by eye. It failed predictably. A sleeveless top and a short sleeve look identical near the shoulder if the only signal is nearby skin. Crossed arms hide both underarms. A rolled long sleeve puts skin far from the actual sleeve fabric. And fashion photography rarely exposes much skin at the sleeve boundary to begin with. The core idea was wrong often enough that no amount of tuning would fix it so I set it aside.

Next I tried pose estimation, using a pretrained MediaPipe model to locate shoulders, elbows, and wrists. I drew a tube along the arm path and treated garment pixels inside it as sleeve. This worked much better than the skin approach. On thirty test images it found sleeves correctly and found nothing on sleeveless garments. I also found and fixed a bug, the first version drew a straight line from shoulder to wrist, so a sharply bent arm caused the tube to miss the bend entirely and point into empty space. Switching to a two-segment path, shoulder to elbow then elbow to wrist, fixed it.

But I deliberately chose not to use pose estimation to generate training labels. It is still a guess built on another model's guess, not ground truth. More importantly, it was solving the wrong dataset's problem. The Kaggle set had no real sleeve annotations and was not really going to work as a final training set. The pose work did prove valuable later, but for placement logic, not for training data generation.

The brief had pointed at Fashionpedia from the start. As I mentioned earlier, I had tried to download it previously and failed; the images came through but the annotations did not. That is why I went to Kaggle first. Once the Kaggle detour proved that no clever inference produces real sleeve ground truth from a dataset that simply does not have it, I tried Fashionpedia again by giving the link to my AI coding assistant and this time both images and annotations downloaded cleanly.

Fashionpedia's ontology includes garment parts as their own labeled categories, including sleeve and collar. The full training set is around three gigabytes. The validation set is about 1,100 images, fully labeled, and released under a usable license. Given what was available to me, working with the smaller validation set was the sensible choice. It is still real Fashionpedia data with identical category definitions and annotation quality; the only difference is volume.

I down sampled the dataset to a smaller subset due to time constraints I kept only images with at least one main garment category and at least one sleeve part labeled. That left 741 images. I split them 630 for training and 111 for validation using a fixed random seed, and I wrote the split to a file so it would be reproducible and immune to accidental leakage on later runs.

Model choice: MobileNetV3-Small frozen, plus a custom decoder

The assessment allowed a compact U-Net from scratch, a small detection model, or a frozen pretrained backbone with a trainable decoder. I chose the third option because of where the parameter budget goes.

A backbone turns a raw image into feature maps. MobileNetV3-Small is pretrained on ImageNet, so it already understands edges, textures, and general visual structure before it sees a single garment photo. Frozen pretrained weights do not count against the two-million parameter cap; only trainable weights do. MobileNetV3-Small's convolutional layers have about 927,000 parameters. By freezing all of them, I get all that pretrained visual knowledge for free, and every parameter I train can go toward the job of turning general features into the specific answer I need: which pixels are body, which are sleeve, and which are background.

The decoder was built from scratch. MobileNetV3-Small processes the image in stages. I tapped three points in that stack: early, when features are close to raw pixels; middle, when they represent larger shapes; and late, when they are abstract and semantic. I chose the specific layer numbers by running a test image through the network, printing the output shape at every layer, and picking three points that gave a good spread between fine detail and broad meaning.

The decoder fuses those three taps starting from the most abstract and working back toward the most detailed, upsizing at each step so the final output aligns pixel for pixel with the fine-detail tap. This is a standard feature pyramid. Deep features know roughly where the garment is and which part is which, but they are blurry about exact edges. Shallow features know exactly where an edge falls, but not what it means. Combining them gives roughly correct meaning placed on precisely correct boundaries.

The entire decoder ended up at about 152,000 trainable parameters, well under 10 percent of the cap. That headroom mattered. It meant I could expand the decoder later without risk of exceeding the limit, and I did.

Training: an honest progression of numbers

The first version used plain cross-entropy loss, treating every pixel equally. It reached a mean IoU of about 0.65. Background scored ~ 0.96 , which is expected. Body scored ~ 0.61 . Sleeve scored only 0.38.

I investigated why sleeve was so weak. Sleeve pixels are a small fraction of the total image, roughly a quarter of the garment's area and a much smaller fraction of the whole photo. A loss function that treats every pixel equally barely notices sleeve mistakes because there are so few of them compared to background. I fixed this by reweighting the loss based on class frequency, and by adding Dice loss, which directly rewards overlapping regions rather than treating pixels as independent guesses. I also doubled the decoder's internal width, since I had so much unused headroom. That combination raised sleeve to ~ 0.45 , body improved slightly, and mean IoU reached ~ 0.68 .

Then I asked a more careful question: was sleeve weak because it is a thin, awkward shape, or was something bigger happening? I measured, for every validation image, how IoU on each class related to the physical area that class occupied. The pattern was clear: body's score depended on garment size even more strongly than sleeve's did. When the garment was small in the frame, every foreground class suffered together. The model was working at too low an input resolution to preserve detail on small, distant garments. I raised the training resolution from 224 to 320 pixels with no other architecture change. Mean IoU moved to ~ 0.69 , with both body and sleeve

improving together, exactly as the size analysis predicted. It also fixed a case where the model had previously predicted nothing at all on a small distant subject.

I tried one more thing: unfreezing the last two backbone layers so a small part of the pretrained network could adapt, with a much smaller learning rate to protect existing knowledge. I ran the full training. Body moved up slightly. Sleeve came back essentially flat, perhaps a hair worse, while the gap between training and validation loss widened in the exact pattern you see when a model starts memorizing instead of generalizing. With only 630 training images, tripling the trainable parameter count was asking for overfitting. I set the change aside. It added real complexity and real risk for no meaningful gain. The honest answer is that sleeve segmentation is genuinely harder given the data available, and pushing further would need more or different training data rather than more tuning.

Placement logic: from segmentation to the business problem

Once the segmentation model existed, the second half was the placement function. The core idea is simple: subtract sleeve pixels from the body prediction so what remains is strictly the torso. Artwork is only ever placed inside that torso area. Because sleeve is subtracted out entirely, artwork can never land on a sleeve or straddle a seam, not because I check for that after the fact, but because the allowed region never includes those pixels in the first place.

The instruction, something like "front, left chest," is parsed for keywords (left, right, centre, chest, front, back) and used to narrow the torso to a specific zone. That zone is measured against the torso's own boundaries in that specific photo, not against the whole image. "Left chest" on a close subject and "left chest" on a distant subject end up in different raw coordinates, but both correctly land on the actual left chest because the zone is computed relative to the detected body, not a fixed canvas point. Once the zone is found, I place the artwork at the point farthest from any zone edge, scaled down if needed to fit safely. If no point in the zone is large enough, the function refuses to place the artwork and explains why.

Pose estimation earned its place back here. While testing chest-zone logic on photos with raised arms, I found a bug: the code defined chest as the top portion of the torso's bounding box, and a raised arm stretches that bounding box upward toward the hand, which quietly shifted "chest" toward the wrist. I reused the pose model from the earlier experiments, not to guess sleeve shape, but to find the actual shoulder line and use it as the reference for where chest begins, regardless of arm position.

I also used the absence of a detected person as a signal that a photo might be a flat lay or hanger shot rather than a worn garment. The segmentation model was trained almost entirely on worn garments and behaves unpredictably on product photography with no person. When no person is

detected, the code falls back to a purely geometric guess based on the garment outline itself. I tested this against an actual flat-lay coat photo, where the first attempt placed a logo on the collar and the fallback moved it onto the actual chest panel.

Testing by hand caught two other issues. One test used "left breast" instead of "left chest," and the code silently misread it as generic centered placement because the keyword list only recognized "chest." I added "breast" as a recognized word, but this reiterates the fact that the instruction parsing is simple keyword matching, not language understanding. Any phrase outside the known vocabulary will silently do the wrong thing.

The other issue came from deliberately hunting for the model's weakest points: a wedding dress photo, an unusual style compared to the training data, got labeled almost entirely as sleeve instead of body. I checked this against an earlier model version to confirm my loss reweighting had not caused it. The earlier version made the exact same mistake in the same spot. This is a genuine gap in training data coverage, not a bug I introduced.

Why I trained on a laptop:

The brief suggested a free Colab or Kaggle GPU session. I trained everything on a regular laptop CPU instead. With the backbone frozen and only a small decoder training on 630 small images, one full epoch took under a minute, and a full 25-epoch run finished in well under half an hour. Setting up a cloud notebook, uploading the dataset, and managing a browser-based environment would likely have taken as long as or longer than the training itself, and would have added friction at the point where time was tightest. A GPU session is described as sufficient, not required. For a model this small, a laptop CPU was sufficient too. Every checkpoint, every number, and every rendered image was produced by code actually run on that machine, which makes reproduction straightforward.

Honest limitations:

This system has real boundaries. The instruction parser is keyword matching, not natural language understanding. The segmentation model was trained almost entirely on worn garments and is unreliable on flat lays or hanger shots without the pose-estimation fallback. And a garment style that is rare in the training data, like the wedding dress, can be confidently misclassified. These are not tuning problems. They are data-coverage problems, and the honest fix is more or different training data rather than more architecture changes.

AI USE DECLARATION:

The architectural decisions, dataset choices, model design, and overall problem-solving approach are entirely my own. I used Claude Code to generate the bulk of the implementation code, which I then reviewed, modified, and approved manually. All written documentation was drafted by me and subsequently refined with AI assistance.